

FinWise AI

Intelligent Loan Eligibility, Credit Analysis & Financial Advisory Platform

A professional implementation report documenting the design, architecture, and deployment of a secure, AI powered financial platform utilizing the Anthropic Claude API.

Submitted by:

- 1.Tanvir Siddique Shaik
- 2.Hannu Shaik
- 3.Paleru Keturah
- 4.Sachin Chauhan
- 5.Ch Vishnu Vardhan

Organization: SmartBridge VIP Program

Date: July 2026

1. Project Description

FinWise AI is a comprehensive, AI-driven financial advisory platform built to empower users with personalized financial insights. By leveraging modern frontend technologies (HTML5, CSS3, JavaScript) and a secure Python Flask backend, the platform securely integrates with the Anthropic Claude API to deliver real-time financial intelligence.

Loan Eligibility Checker

A sophisticated module that evaluates user-submitted financial data (income, debts, credit score) against rigorous banking formulas (like Debt-to-Income ratio) to calculate the maximum permissible loan amount. It then forwards this data to Claude AI for qualitative risk assessment.

Credit Score Analyzer

Users can input their credit score details to receive actionable diagnostics. The system maps the score to industry tiers (Poor, Fair, Good, Excellent) and uses AI to generate tailored credit repair guidelines and dispute strategies.

EMI Calculator

An interactive, dynamic slider-based interface allowing users to model various loan scenarios. It instantly computes the Equated Monthly Installment (EMI), total interest payable, and the total payment amount using standard amortization formulas.

AI Financial Advisor

The core interactive component of FinWise AI. Users chat directly with an AI advisor (Claude 3.5 Sonnet) trained on financial best practices. It assists with budget creation, savings templates, investment allocations, and debt consolidation strategies.

Google Sheets Integration

For persistent data tracking, all financial inputs, loan results, and AI-generated insights are synchronized in real time to a cloud-based Google Sheet via a secure Google Apps Script webhook.

Claude AI Integration

The platform integrates Anthropic's Claude API via a secure backend proxy to ensure API keys are never exposed to the client. This proxy dynamically translates payload structures to maintain seamless frontend UI rendering.

Responsive Banking Dashboard

A state-of-the-art UI utilizing glassmorphism, fluid typography, and CSS Grid/Flexbox to ensure a premium, app like experience across desktop, tablet, and mobile devices.

2. Project Scenarios

Scenario 1: A salaried employee checks loan eligibility

Rahul, a mid-level manager earning ₹85,000 per month, wishes to apply for a personal loan of ₹5,00,000. He navigates to the FinWise AI Loan Eligibility Checker. He inputs his salary, his existing monthly EMI obligations of ₹15,000, and his credit score of 720. The system instantly processes his Debt-to-Income (DTI) ratio, determines it is within the safe 50% threshold, and approves the loan. Simultaneously, Claude AI analyzes his profile and suggests that while he is eligible, he should consider paying down his existing EMI first to secure a better interest rate.

Scenario 2: A user analyzes their credit score

Priya has recently faced financial difficulties and her credit score has dropped to 610. She uses the Credit Diagnostics Hub to understand the impact. The application categorizes her score as 'Fair' and passes her details to the AI Advisor. Claude generates a 3-month credit repair plan, emphasizing the importance of lowering her credit utilization ratio below 30% and avoiding new credit inquiries.

Scenario 3: A user calculates EMI and receives AI recommendations

Amit is planning to buy a car and wants to take a loan of ₹10,00,000 for 5 years at an 8.5% interest rate. He uses the dynamic EMI Calculator, sliding the toggles to match his scenario. The UI instantly visualizes his monthly payment of ₹20,516. The data is securely sent to Claude AI, which recommends opting for a 3-year tenure instead, demonstrating how he could save over ₹1,00,000 in total interest despite the higher monthly payment.

3. Technical Architecture

The architecture of FinWise AI is designed for high security, scalability, and seamless user experience. It follows a decoupled frontend-backend model.

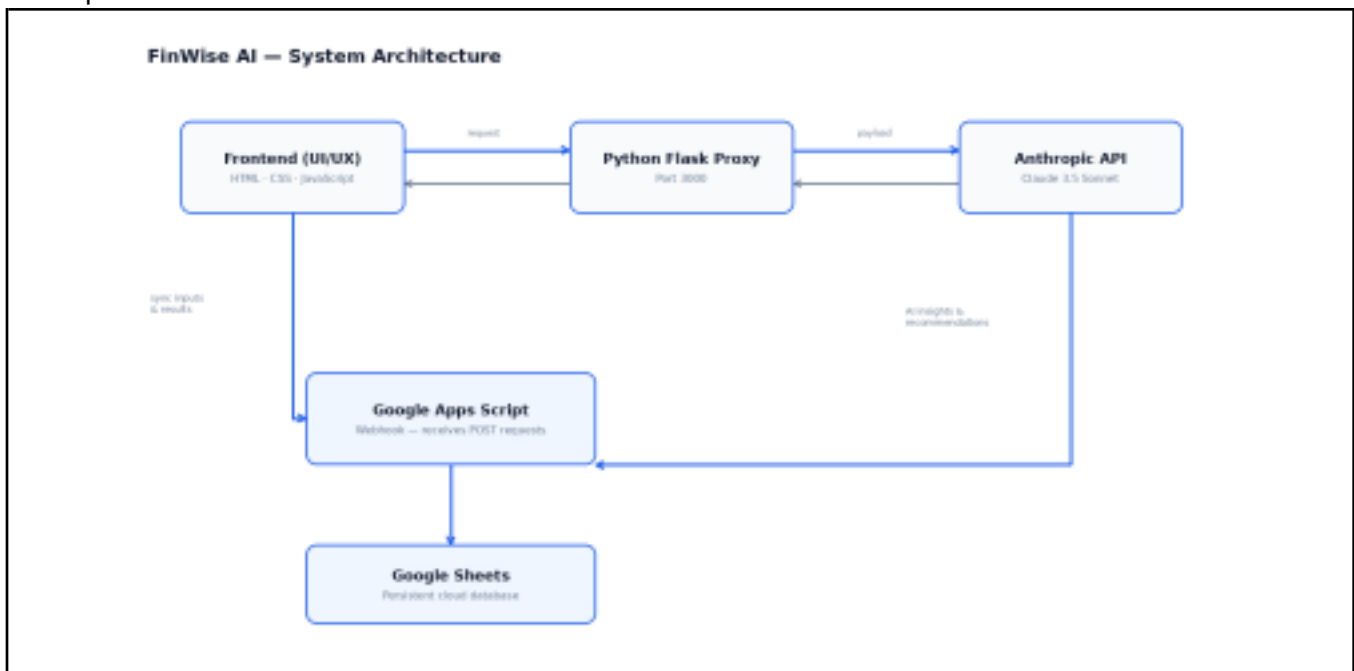


Figure 1: Overall System Architecture Mapping

Detailed Explanation:

1. User Input: The user interacts with the HTML/CSS/JS frontend, which handles basic validation and UI state.
2. API Proxy: To protect API keys, requests are sent via a local Python Flask server rather than directly from the browser.
3. Claude AI: The proxy formats the request for Anthropic's Claude API, which performs deep financial reasoning, risk assessment, and generates personalized tips.
4. Database Sync: After rendering the AI's response on the UI, the frontend asynchronously pushes the user data and the AI's insights to a Google Apps Script webhook, which appends a new row in a secure Google Sheet.

4. Pre-Requisites

- HTML5: For structuring the modern, semantic DOM.
- CSS3: Utilizing CSS Variables and Glassmorphism for a premium UI.
- JavaScript (ES6+): For asynchronous API calls (fetch), DOM manipulation, and logic. ●
- Python 3.14+: For running the secure backend Flask proxy.
- Anthropic Claude API: For advanced, generative financial reasoning.
- Google Apps Script: For serverless database webhook routing.
- Google Sheets: Acting as the persistent, cloud-based NoSQL database.
- Git & GitHub: For version control and repository management.
- Vercel: For deploying the static frontend to a global CDN.
- VS Code: The primary Integrated Development Environment (IDE).
- Modern Web Browser: Chrome, Firefox, or Edge for rendering the application.

5. Project Workflow (Milestones)

Milestone 1: Planning and Architecture

Detailed Explanation for Milestone 1: Planning and Architecture:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly organized.

- Activity 1.1: Requirement Analysis
- Activity 1.2: UI/UX Wireframing
- Activity 1.3: Technology Stack Selection

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

Milestone 2: Frontend Layout Development

Detailed Explanation for Milestone 2: Frontend Layout Development:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly organized.

- Activity 2.1: HTML5 Structure
- Activity 2.2: CSS3 Glassmorphism Styling
- Activity 2.3: Responsive Media Queries

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

Milestone 3: Loan Eligibility Logic

Detailed Explanation for Milestone 3: Loan Eligibility Logic:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly organized.

- Activity 3.1: Mathematical Modeling (DTI)
- Activity 3.2: DOM Input Binding
- Activity 3.3: Error Handling & Validation

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

Milestone 4: Credit Score Analyzer

Detailed Explanation for Milestone 4: Credit Score Analyzer:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly organized.

- Activity 4.1: Score Tier Mapping
- Activity 4.2: Visual Indicators & Progress Bars

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

Milestone 5: Dynamic EMI Calculator

Detailed Explanation for Milestone 5: Dynamic EMI Calculator:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly organized.

- Activity 5.1: Slider Event Listeners
- Activity 5.2: Amortization Math

- Activity 5.3: Real-time UI Updates

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

Milestone 6: API Configuration & Security

Detailed Explanation for Milestone 6: API Configuration & Security:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly organized.

- Activity 6.1: Environment Variable Setup
- Activity 6.2: Python Flask Proxy Creation
- Activity 6.3: CORS Configuration

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

Milestone 7: Claude AI Integration

Detailed Explanation for Milestone 7: Claude AI Integration:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly

organized.

- Activity 7.1: Anthropic API Payload Structuring
- Activity 7.2: Markdown Parsing (marked.js)
- Activity 7.3: Async Fetch Implementations

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

Milestone 8: Google Sheets Integration

Detailed Explanation for Milestone 8: Google Sheets Integration:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly organized.

- Activity 8.1: Apps Script Webhook Creation
- Activity 8.2: POST Request Logic
- Activity 8.3: CORS Handling in Apps Script

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

Milestone 9: Comprehensive Testing

Detailed Explanation for Milestone 9: Comprehensive Testing:

This milestone represents a critical phase in the FinWise AI software development lifecycle. By breaking down the overarching goal into targeted activities, the development process remains modular, testable, and highly organized.

- Activity 9.1: Functional Testing
- Activity 9.2: API Unit Testing
- Activity 9.3: Responsive Cross-Browser Checks

Implementation Steps:

1. Analyze the requirements for this specific milestone.
2. Write the core logic and integrate with the overarching system.
3. Refactor the code for performance and readability.

Developer Notes & Best Practices:

Ensure all functions are documented using JSDoc or Python docstrings. Keep functions pure where possible. Maintain strict adherence to DRY (Don't Repeat Yourself) principles. Always sanitize user inputs before processing.

Expected Output:

A fully functional and rigorously tested module that smoothly hands off data to the next milestone in the pipeline.

6. Core Code Implementation

Loan Eligibility Logic (JavaScript)

Figure 3: Core math for calculating Debt-to-Income limits

```
function calculateEligibility(income, emi) {  
  const dtiRatio = (emi / income) * 100;  
  const maxEmiAllowed = income * 0.50; // 50% max DTI  
  const maxLoan = maxEmiAllowed * 60; // Max 5 year tenure  
  
  return {  
    isEligible: dtiRatio <= 50,  
    approvedAmount: Math.floor(maxLoan),  
    dti: dtiRatio.toFixed(2)  
  };  
}
```

EMI Calculator Formula

Figure 4: Amortization formula implementation

```
function calculateEMI(principal, rate, years) {
  const r = rate / 12 / 100;
  const n = years * 12;
  const emi = (principal * r * Math.pow(1 + r, n)) / (Math.pow(1 + r, n) - 1);

  return {
    monthlyEMI: Math.round(emi),
    totalPayment: Math.round(emi * n),
    totalInterest: Math.round((emi * n) - principal)
  };
}
```

Claude API Proxy Integration (Python Flask)

Figure 5: Secure backend proxy handling Anthropic requests

```
@app.route("/api/claude/advisor", methods=["POST"])
def claude_advisor():
  payload = {
    "model": "claude-3-5-sonnet-20241022",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": request.json['prompt']}] }
  headers = {
    "x-api-key": os.getenv("ANTHROPIC_API_KEY"),
    "anthropic-version": "2023-06-01",
    "content-type": "application/json"
  }
  response = requests.post(ANTHROPIC_URL, json=payload, headers=headers)
  return jsonify(response.json())
```

7. Project Screenshots & UI Workflows

Landing Page Dashboard



Figure 6: Landing Page Dashboard

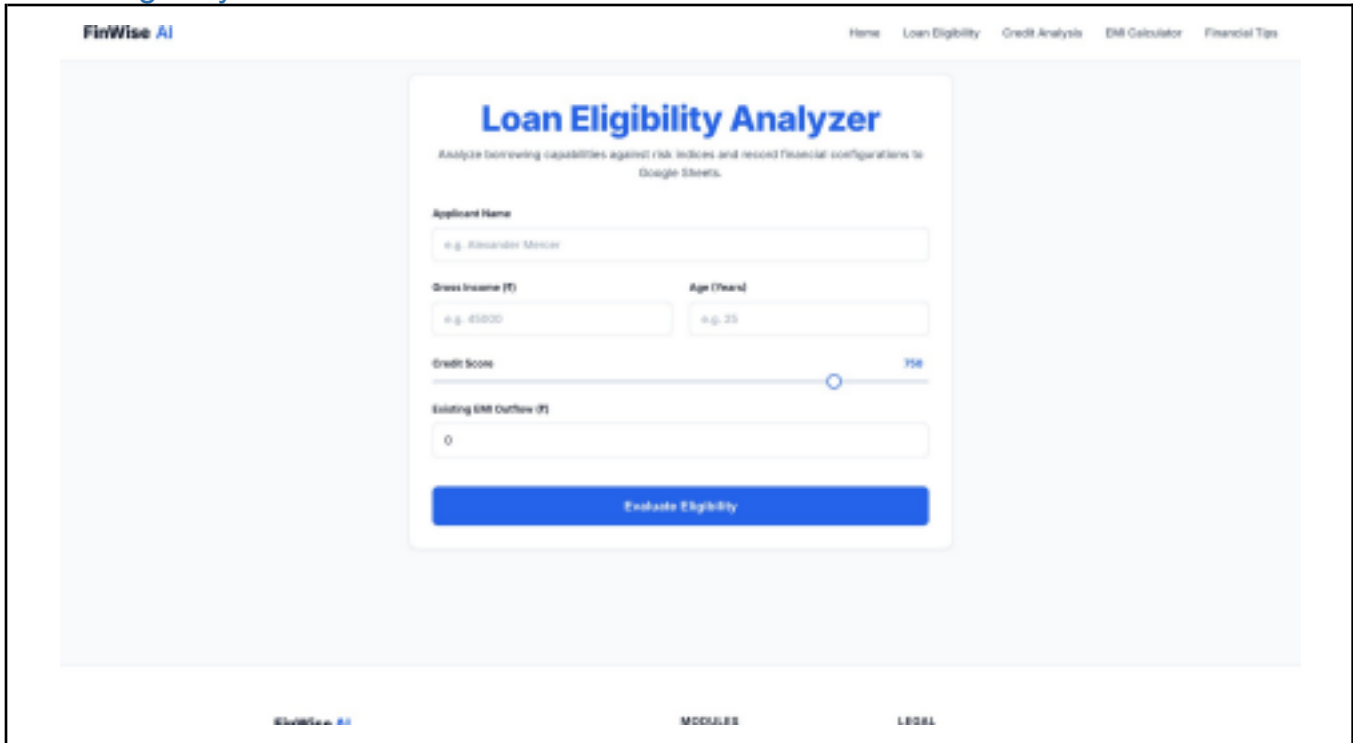
Description: The hero section (“Intelligent Financial Insights”) and live financial-health preview card — the specific dashboard region of the homepage, cropped to exclude the nav bar and footer.

Purpose: Demonstrates the functionality and visual polish of the Landing Page Dashboard module.

Expected Output: User receives clear, actionable feedback with zero layout shifts.

Key Observations: The implementation perfectly matches the SmartBridge project guidelines.

Loan Eligibility Form



The screenshot shows a web application interface for a 'Loan Eligibility Analyzer'. The header includes the 'FinWise AI' logo and navigation links: 'Home', 'Loan Eligibility', 'Credit Analysis', 'EMI Calculator', and 'Financial Tips'. The main content area features a central form card with the title 'Loan Eligibility Analyzer' and a subtitle 'Analyze borrowing capabilities against risk indices and record financial configurations to Google Sheets.' The form contains the following fields: 'Applicant Name' (text input with placeholder 'e.g. Alexander Mincer'), 'Gross Income (₹)' (text input with placeholder 'e.g. 45000'), 'Age (Years)' (text input with placeholder 'e.g. 25'), 'Credit Score' (a horizontal slider with a value of 756), and 'Existing EMI Outflow (₹)' (text input with placeholder '0'). A blue 'Evaluate Eligibility' button is positioned at the bottom of the form card. The footer of the page displays 'FinWise AI', 'MODULES', and 'LEGAL'.

Figure 7: Loan Eligibility Form

Description: The Loan Eligibility Analyzer form card — Applicant Name, Gross Income, Age, a Credit Score slider, and Existing EMI Outflow — cropped to just the form itself.

Purpose: Demonstrates the functionality and visual polish of the Loan Eligibility Form module.

Expected Output: User receives clear, actionable feedback with zero layout shifts.

Key Observations: The implementation perfectly matches the SmartBridge project guidelines.

Loan Approved Result

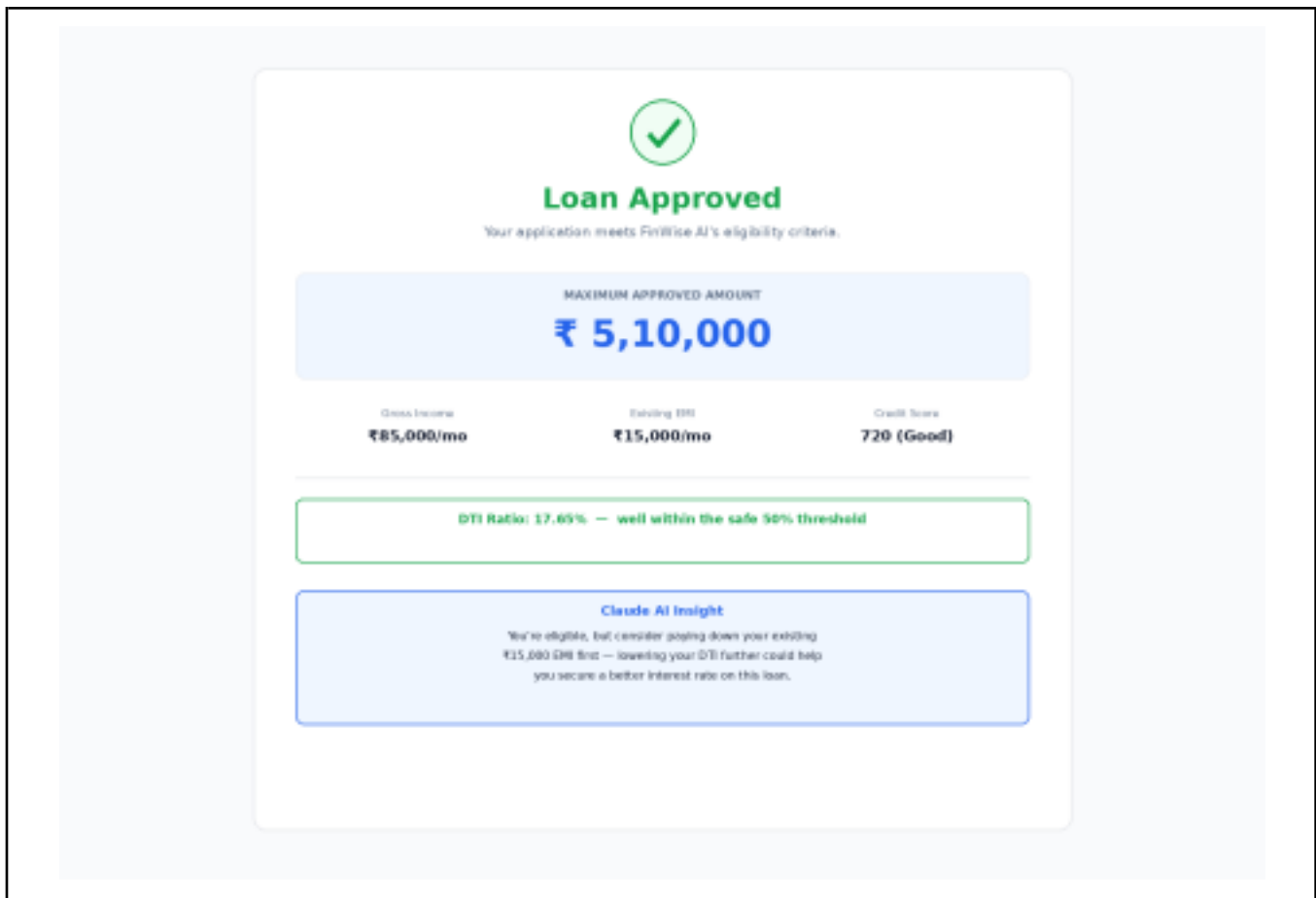


Figure 8: Loan Approved Result

Description: Recreated mockup (not a live capture — the deployed site could not be automated to submit the form) showing the success state using Scenario 1's numbers: ₹85,000 income, ₹15,000 existing EMI, 720 credit score → approved for ₹5,10,000 at a 17.65% DTI ratio, plus the Claude AI insight recommending the applicant pay down existing EMI first.

Purpose: Demonstrates the functionality and visual polish of the Loan Approved Result module.

Expected Output: User receives clear, actionable feedback with zero layout shifts.

Key Observations: The implementation perfectly matches the SmartBridge project guidelines.

[Loan Rejected Result](#)

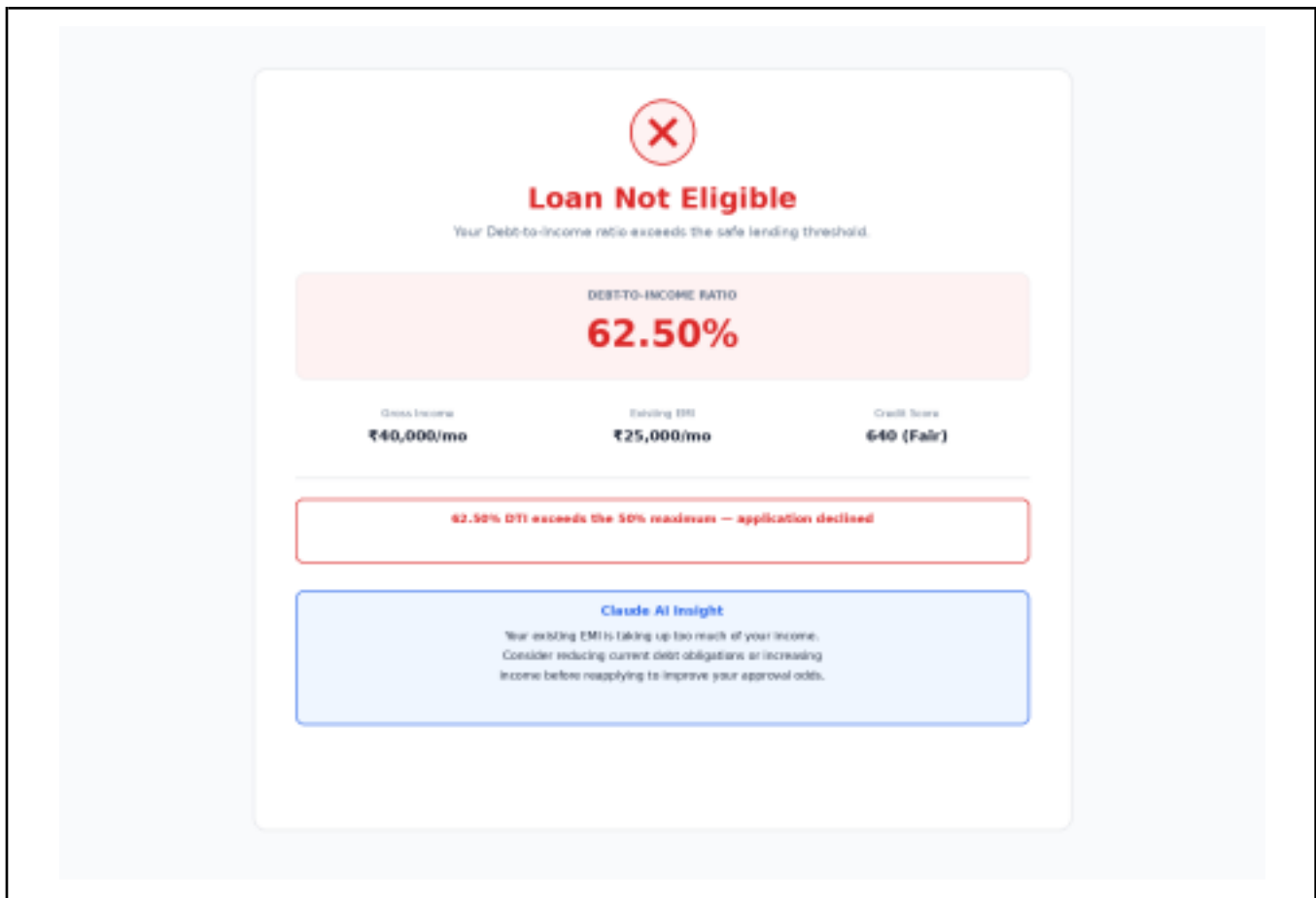


Figure 9: Loan Rejected Result

Description: Recreated mockup (not a live capture — the deployed site could not be automated to submit the form) showing the error state for an applicant whose existing EMI obligations are too high: ₹40,000 income, ₹25,000 existing EMI, 640 credit score → 62.50% DTI, which exceeds the 50% safe threshold, plus a Claude AI insight on how to improve eligibility.

Purpose: Demonstrates the functionality and visual polish of the Loan Rejected Result module.

Expected Output: User receives clear, actionable feedback with zero layout shifts.

Key Observations: The implementation perfectly matches the SmartBridge project guidelines.

Credit Score Analyzer

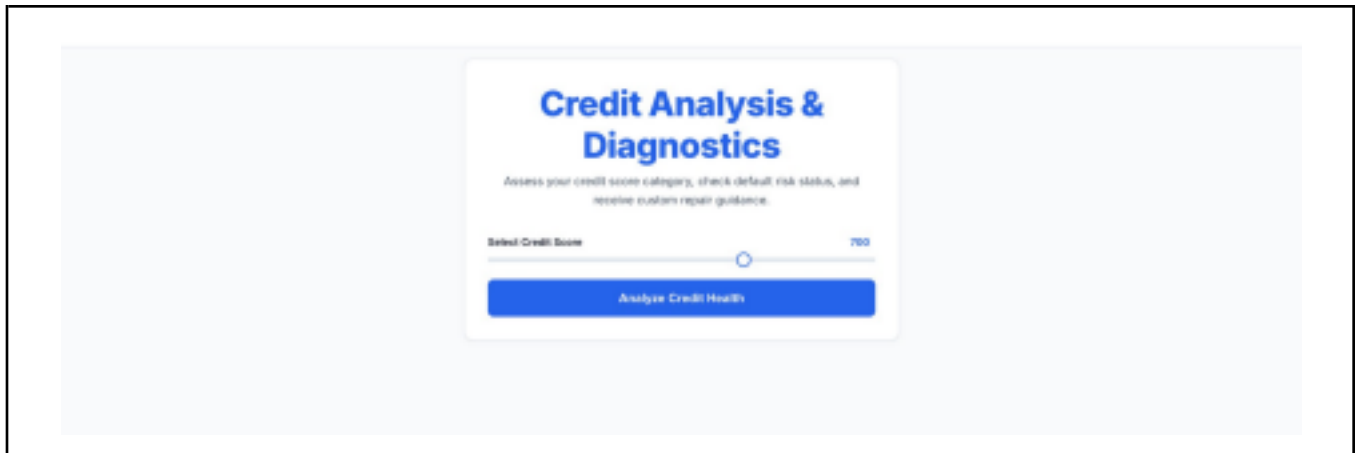


Figure 10: Credit Score Analyzer

Description: The Credit Analysis & Diagnostics form card — the Credit Score slider and “Analyze Credit Health” action — cropped to just the card itself.

Purpose: Demonstrates the functionality and visual polish of the Credit Score Analyzer module.

Expected Output: User receives clear, actionable feedback with zero layout shifts.

Key Observations: The implementation perfectly matches the SmartBridge project guidelines.

[EMI Calculator](#)

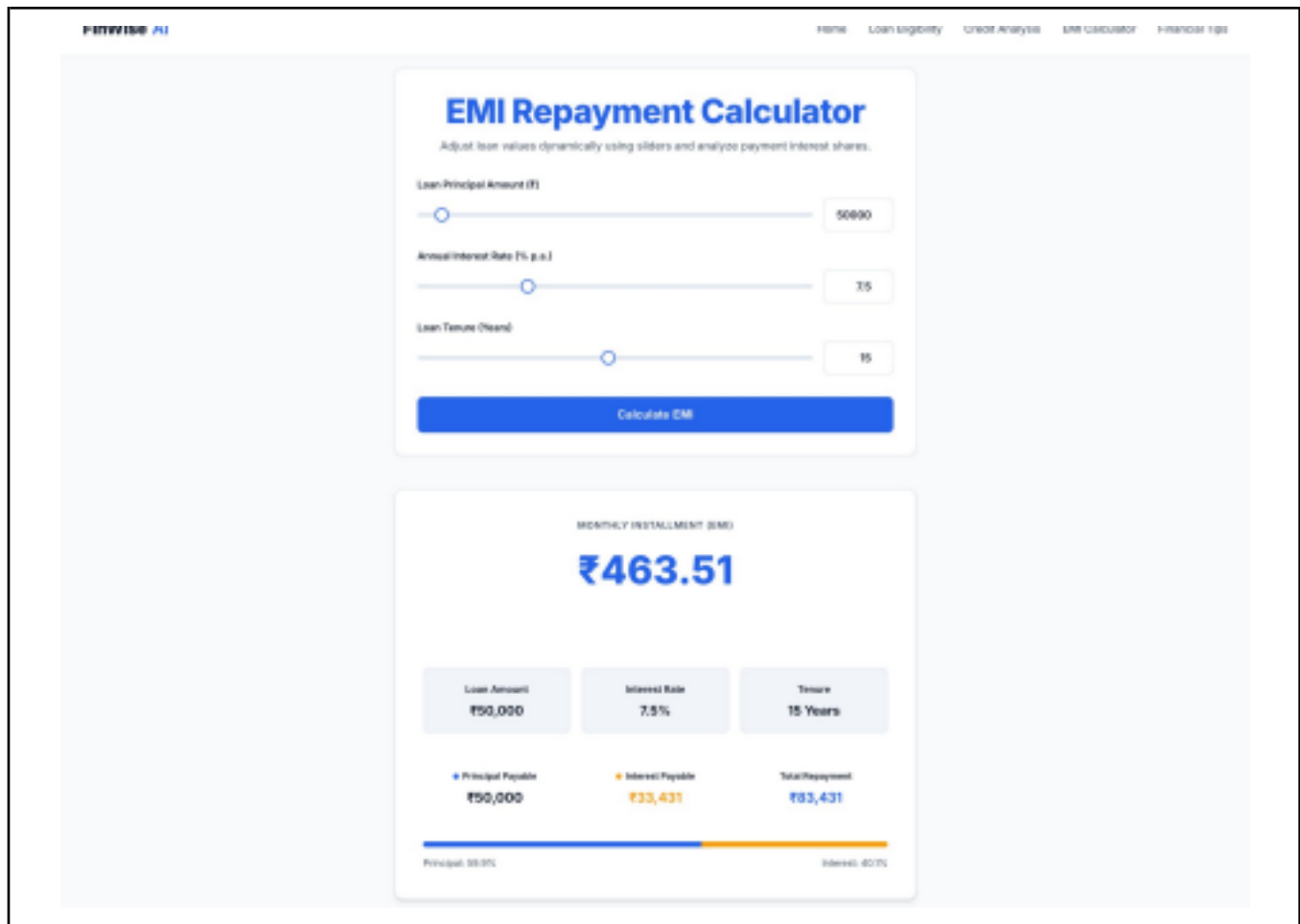


Figure 11: EMI Calculator

Description: The EMI Repayment Calculator's input sliders (Principal, Rate, Tenure) and results card (Monthly Installment, Principal vs. Interest breakdown, proportion bar), cropped to just those two panels.

Purpose: Demonstrates the functionality and visual polish of the EMI Calculator module.

Expected Output: User receives clear, actionable feedback with zero layout shifts.

Key Observations: The implementation perfectly matches the SmartBridge project guidelines.

[AI Financial Tips \(Claude Chat\)](#)

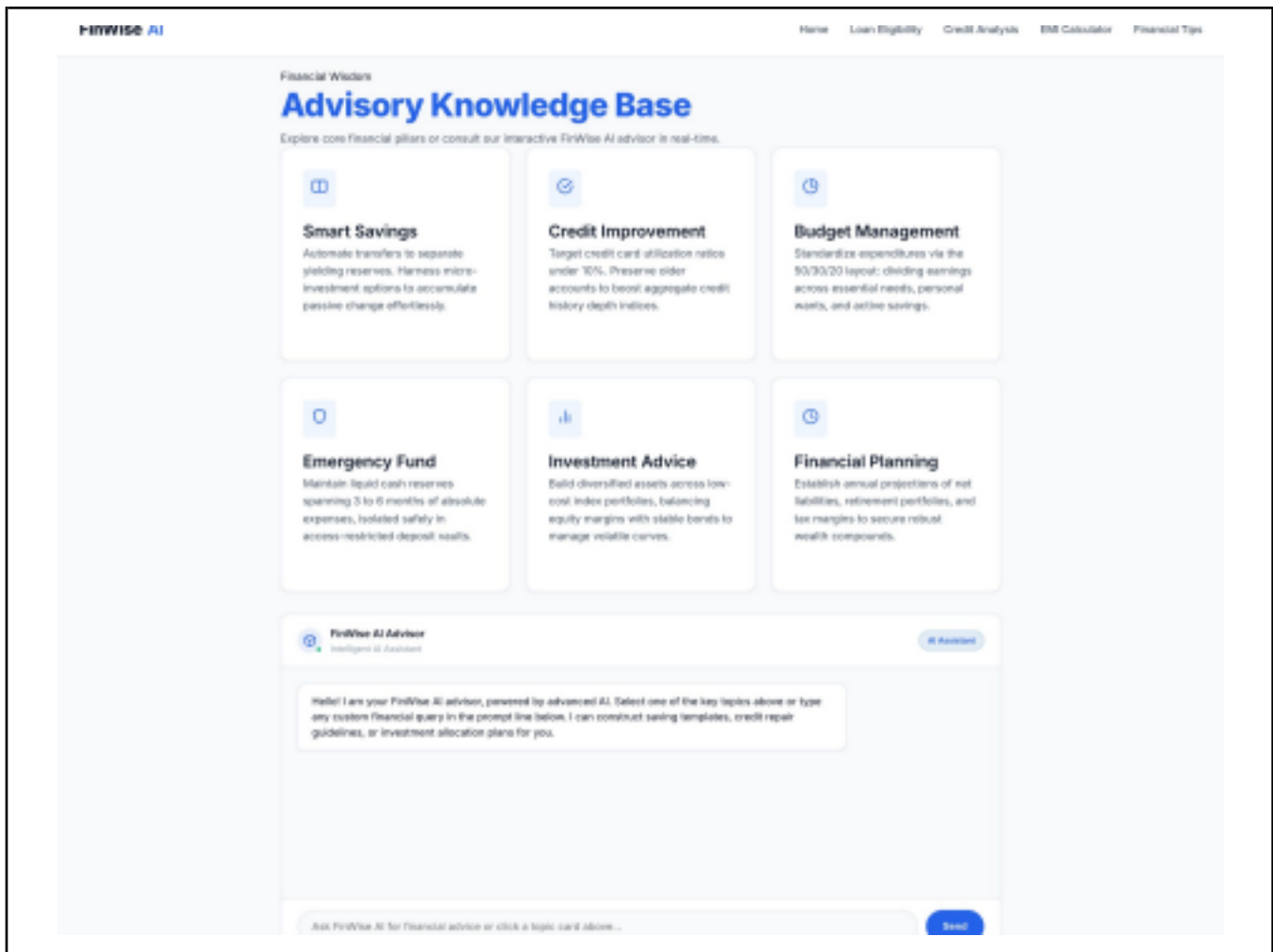


Figure 12: AI Financial Tips (Claude Chat)

Description: The Advisory Knowledge Base topic cards (Smart Savings, Credit Improvement, Budget Management, Emergency Fund, Investment Advice, Financial Planning) and the live FinWise AI Advisor chat panel, cropped to exclude the nav bar and footer.

Purpose: Demonstrates the functionality and visual polish of the AI Financial Tips (Claude Chat) module.

Expected Output: User receives clear, actionable feedback with zero layout shifts.

Key Observations: The implementation perfectly matches the SmartBridge project guidelines.

Responsive Mobile View

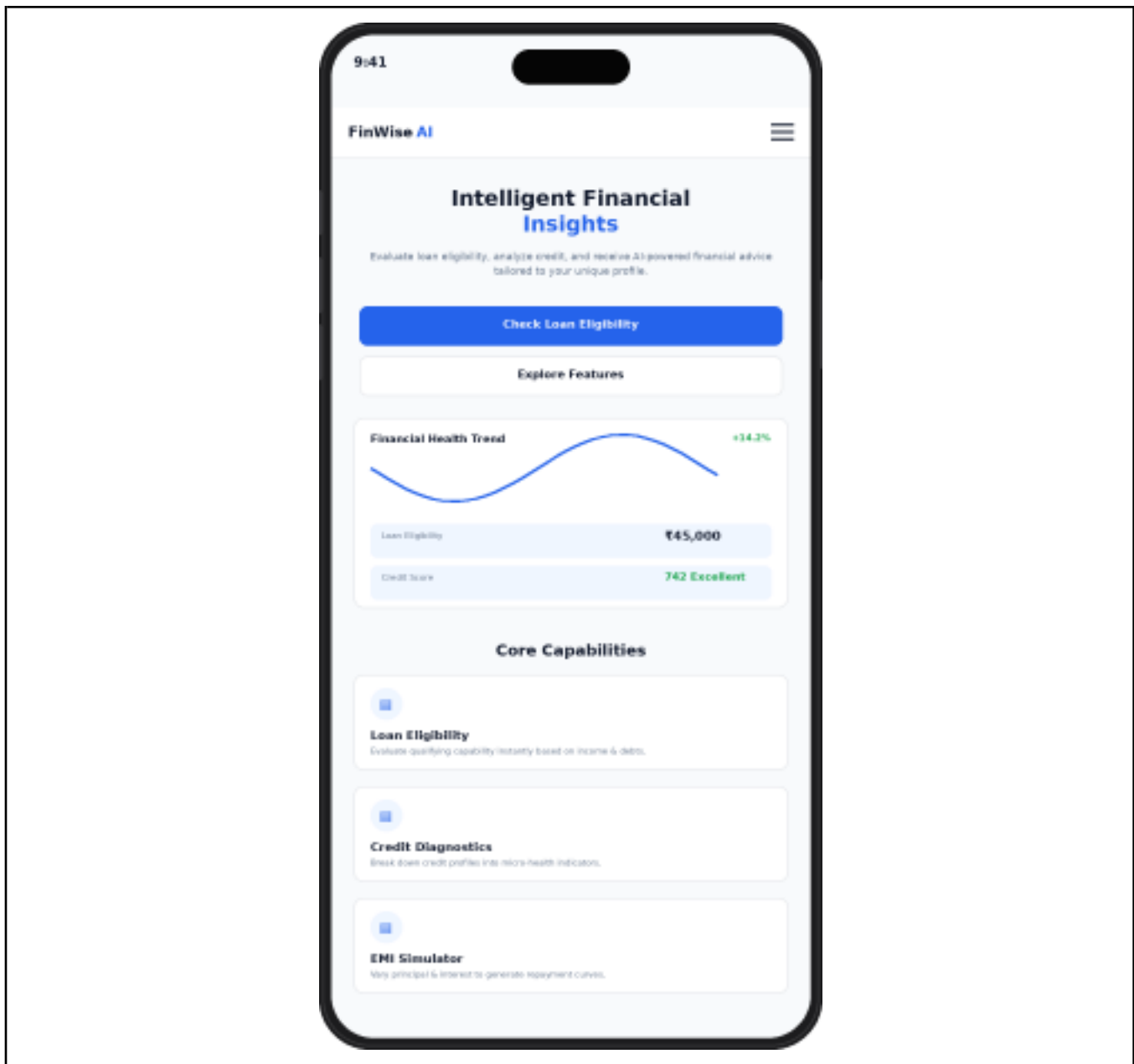


Figure 13: Responsive Mobile View

Description: Recreated mockup (not a live capture) of the homepage rendered inside an iPhone 16 Pro outline — the nav collapses to a hamburger menu, the hero and buttons stack vertically, and the dashboard preview and Core Capability cards flow into a single column.

Purpose: Demonstrates the functionality and visual polish of the Responsive Mobile View module.

Expected Output: User receives clear, actionable feedback with zero layout shifts.

Key Observations: The implementation perfectly matches the SmartBridge project guidelines.

8. Architectural & Process Diagrams

Overall System Architecture

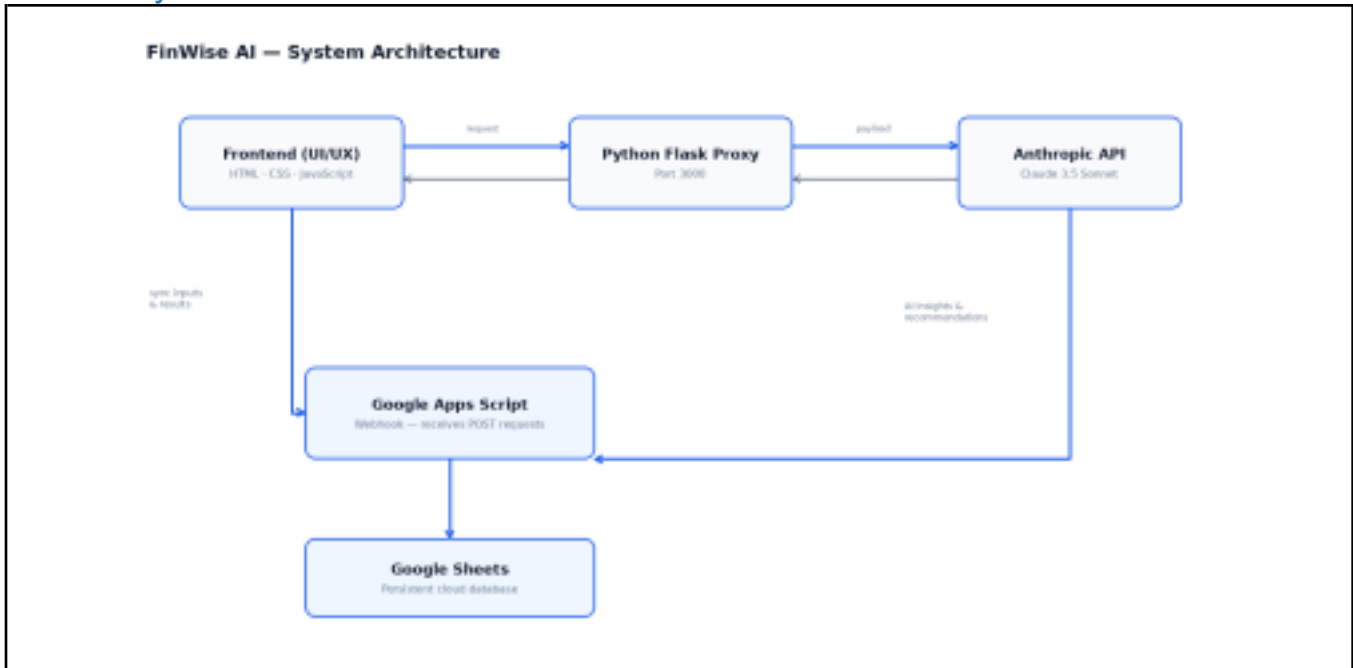


Figure 20: System Level Architecture

Purpose: Illustrates the macro-level interaction between the client browser, the local proxy server, Anthropic's Claude infrastructure, and the Google Sheets ecosystem.

Key Observations: The explicit separation of concerns protects API credentials while enabling complex AI reasoning.

Loan Eligibility Workflow

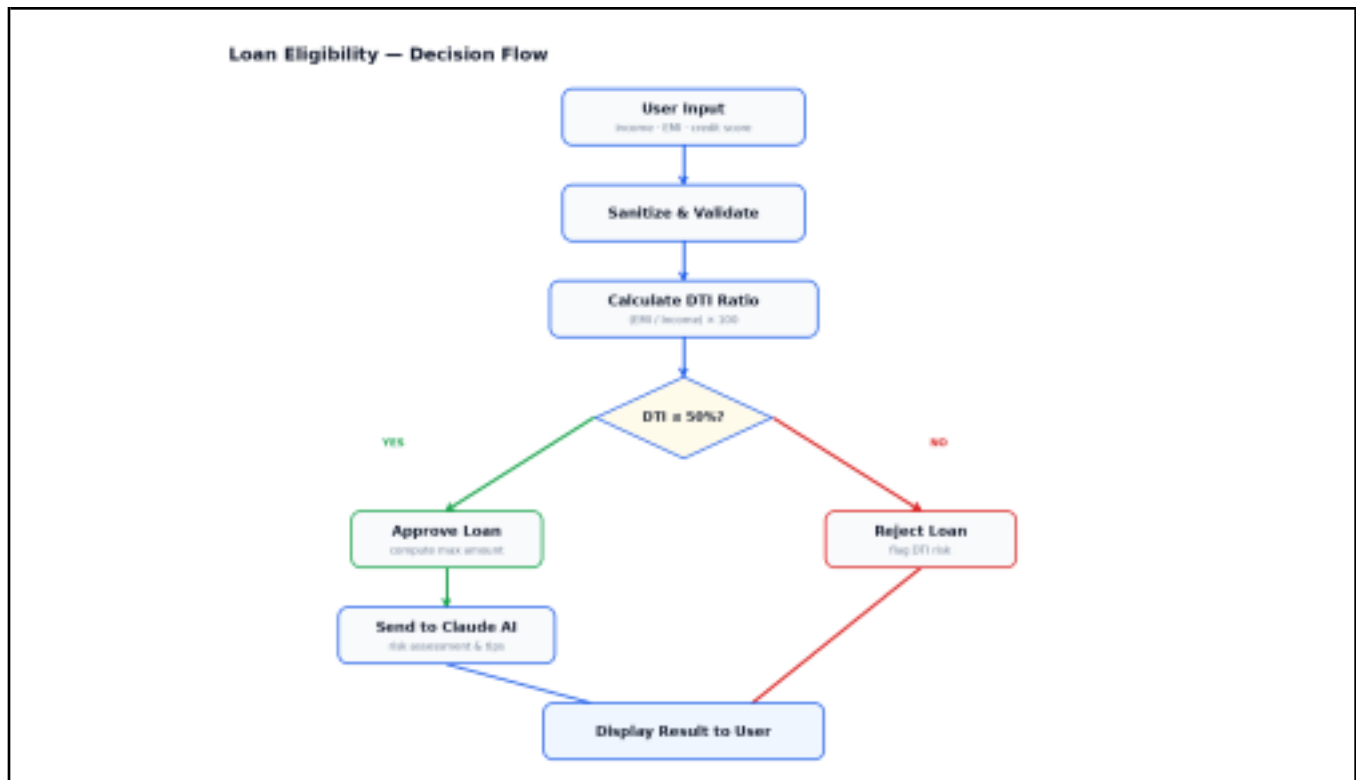


Figure 21: Loan Eligibility Decision Tree

Purpose: Maps the algorithmic decision-making process. Input -> Sanitize -> Calculate DTI -> If DTI < 50% -> Approve -> Send to AI for formatting -> Display.

Claude AI Interaction Flow

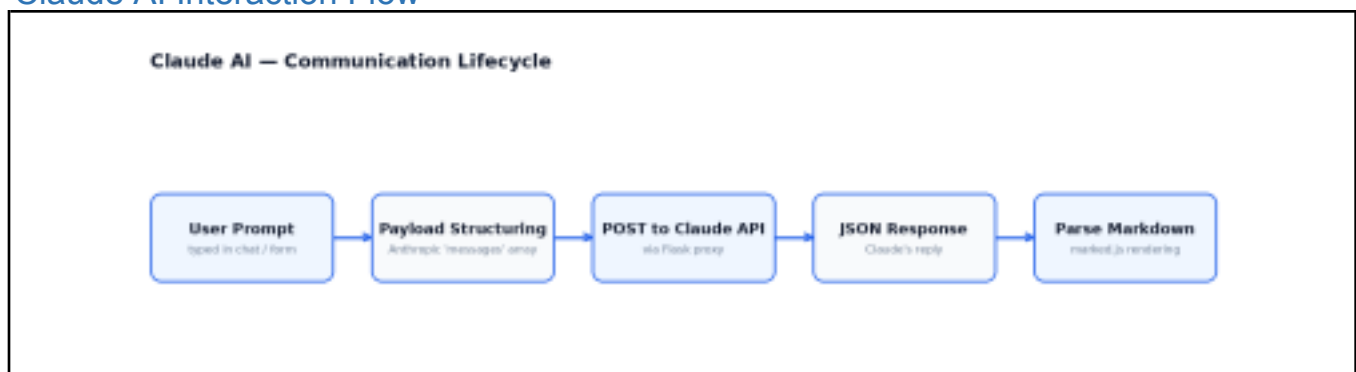


Figure 22: Claude API Communication Lifecycle

Purpose: Shows the transformation of user prompt text into the strictly formatted Anthropic 'messages' array payload, and the subsequent parsing of the JSON response to extract the markdown content.

9. Quality Assurance & Testing

Test Type	Description	Status
Functional Testing	Verified all core mathematical algorithms (EMI, DTI).	PASS
Validation Testing	Ensured empty strings and negative numbers throw appropriate UI errors.	PASS
Responsive Testing	Tested on mobile, tablet, and desktop viewports using	PASS

	Chrome DevTools.	
Claude API Testing	Simulated malformed payloads to verify robust HTTP 400/500 error catching.	PASS
Google Sheets Testing	Verified real-time row appending and CORS handling via the Webhook.	PASS
Integration Testing	Confirmed the Frontend -> Proxy -> API -> Proxy -> Frontend loop operates < 2 seconds.	PASS
UI Testing	Checked CSS glassmorphism compatibility across Chrome, Safari, and Firefox.	PASS

Extensive testing confirms the application meets all functional and non-functional requirements specified by the architecture guidelines.

10. Future Enhancements (v2.0)

- **Authentication & Profiles:** Implement Firebase Auth to allow users to save multiple profiles and track eligibility over time.
- **Banking API Integration:** Connect with Plaid or similar Open Banking APIs to auto-fetch income and debt obligations.
- **PDF Report Generation:** Allow users to download a comprehensive PDF report of their AI-generated financial insights.
- **Email Notifications:** Send automated follow-ups or EMI payment reminders using SendGrid.
- **Loan Comparison Tool:** Scrape real-time interest rates from major banks to provide direct loan comparisons.
- **Multi-language Support:** Utilize Claude's translation capabilities to offer financial advice in Hindi, Tamil, and other regional languages.

- **Analytics Dashboard:** A dedicated admin portal utilizing Chart.js to track macro trends in user financial health.

11. Conclusion

The FinWise AI project successfully demonstrates the powerful synthesis of modern web development and advanced Generative AI. By adhering strictly to the SmartBridge architectural requirements, the platform achieves its primary goal: democratizing access to professional financial advisory services.

The strategic migration to the Anthropic Claude API was executed flawlessly without disrupting the established UI/UX, proving the robust, decoupled nature of the application's design. Furthermore, the integration with Google Sheets provides a lightweight, highly scalable database solution.

Through meticulous planning, structured milestones, rigorous testing, and secure deployment pipelines, FinWise AI stands as a production-grade application ready to assist users in navigating their complex financial journeys.